Alexandria University

Faculty of Science

Department
Mathematics and Computer Science

Banking Risk Analysis: EDA, Power BI and Logistic Regression

By: Mina Samir Mounir Gaballah Abadir - 20211442470

Supervised By: Dr/ Nesrine Nabil

2025

Table of Contents

Chapter 1: Introduction & Project Overview

1.1 Background

1.2 Problem Statement

1.3 Overview of the Dataset

1.4 Objective of the Project

1.5 Structure of the Project

1.6 Theoretical Foundation

Chapter 2: Theoretical Foundation of Logistic Regression

2.1 Introduction to Binary Classification

2.2 The Logistic Regression Model

2.2.1 Logistic Function (Sigmoid Function)

2.2.2 Odds and Log-Odds

2.2.3 Odds Ratio

2.3 Comparison with Linear Regression

2.4 Assumptions of Logistic Regression

2.5 Model Evaluation Metrics

2.6 Applications of Logistic Regression

2.7 Conclusion

Chapter 3: Exploratory Data Analysis (EDA)

3.1 Introduction to Exploratory Data Analysis

3.2 Data Cleaning and Preprocessing

3.2.1 Missing Data

3.2.2 Duplicate Data

3.2.3 Data Type Verification

3.3 Visualizing the Data and Gaining Insights

3.3.1 Distribution of Income Band

3.3.2 Risk Weighting Distribution

3.3.3 Nationality Distribution

3.3.4 Income Band by Nationality

3.3.5 Correlation Matrix

3.3.6 Scatter Plot between Bank Deposits and Saving Accounts

3.4 Conclusion of EDA

Chapter 4: Power BI Modeling and Visualization

4.1 Introduction to Power BI

4.1.1 What is Power BI?

4.1.2 Why Power BI is Important for Data Analysis

4.1.3 Key Features of Power BI

4.2 Introduction to Data Modeling in Power BI

4.2.1 What is Data Modeling in Power BI?

4.2.2 Fact Tables and Dimension Tables

4.2.3 Granularity in Data Modeling

4.2.4 Relationships in Power BI

4.2.5 Key Data Modeling Concepts in Power BI

4.3 The Data Model for Our Dataset

4.3.1 Overview of the Dataset

4.3.2 Modeling the Data in Power BI

4.3.3 Relationships in the Data Model

4.3.4 Granularity of the Data

4.3.5 Measures and DAX Calculations

4.4 Power BI Visualizations and Insights

4.4.1 Overview of the Dashboards

4.4.2 Insights and Key Metrics

4.5 Conclusion

4.5.1 Summary of Insights

4.5.2 Future Analysis or Recommendations

Chapter 5: Model Implementation and Evaluation

5.1 Introduction to Logistic Regression

5.1.1 Understanding class_weight='balanced' in Logistic Regression

5.2 Model Training and Evaluation

5.2.1 Training the Logistic Regression Model

5.2.2 Model Evaluation

5.3 Interpretation of Model Coefficients

5.4 ROC Curve and AUC Score

5.5 Conclusion and Insights

5.5.1 Summary of Insights

5.5.2 Recommendations

Chapter 1

Introduction & Project Overview

1.1 Background

In the modern banking sector, one of the most critical tasks is assessing the **financial risk** posed by individual clients. With the increasing complexity of financial transactions, understanding and predicting **high-risk clients** has become paramount for mitigating losses and optimizing risk management strategies. High-risk clients are more likely to default on loans, delay payments, or engage in other behaviors that could lead to financial instability for the bank. Consequently, accurate classification and early identification of these clients allows financial institutions to better allocate resources, adjust their credit policies, and minimize losses.

As financial services become more accessible, banks need to assess client risk efficiently. Traditional methods of **credit scoring** and **risk assessment** are often not sufficient in dealing with the complexity of modern financial data. This project uses **data analytics** and **machine learning** to develop a **predictive model** capable of classifying clients into **high-risk** and **low-risk** categories. The model uses financial and demographic features of clients, leveraging **Logistic Regression**, a statistical technique well-suited for binary classification.

1.2 Problem Statement

The ability to predict whether a client is **high-risk** or **low-risk** is of significant importance for banks and financial institutions. High-risk clients are a source of potential loss for financial institutions, as they are more likely to default on loans or miss payments, leading to financial instability for the bank. Conversely, denying credit to potential customers who are capable of repaying loans can result in missed opportunities for profit.

Accurately classifying clients into these categories is a critical function for **credit risk management**, enabling banks to implement strategies to **mitigate financial risks** and optimize the allocation of resources. Without an effective model for identifying **high-risk clients**, banks may either extend too much credit to risky

clients or reject clients who could repay their debts, affecting both profitability and customer satisfaction.

This project aims to address this issue by developing a **Logistic Regression model** that classifies clients based on their likelihood of being high-risk. The goal is to leverage demographic and financial data to make predictions that can help financial institutions make informed decisions about **loan approvals** and **credit extensions**.

1.3 Overview of the Dataset

The dataset used in this project consists of a range of features representing the **financial profiles** and **demographic characteristics** of clients. It is structured to provide a holistic view of each client, capturing data that may influence their **creditworthiness** and likelihood of being classified as **high-risk** or **low-risk**.

Key features in the dataset include:

- **Age**: The client's age, which may correlate with their financial stability, maturity, and ability to handle debt.
- **Income**: The client's **annual income**, which is often a good indicator of their ability to repay loans.
- **Credit Card Balance**: The total **outstanding balance** on the client's credit card, which is a key factor in assessing financial stress.
- **Bank Loans**: The total loan balance of the client, which can indicate their financial exposure and potential strain.
- **Risk Weighting**: This is the target variable for this project. It classifies clients into two categories: **High-Risk (1)** and **Low-Risk (0)**.

The dataset consists of several hundred records of clients, each with the variables mentioned above. The features will be used to predict whether a client is high-risk or low-risk.

1.4 Objective of the Project

The main objective of this project is to apply **Logistic Regression** to predict the **risk level** of clients based on their demographic and financial data. Specifically, the project will:

- **Classify clients** into **high-risk** or **low-risk** categories based on features like age, income, credit card balance, and loan amounts.
- Use the **Logistic Regression** model to estimate the **probability** that a given client will fall into the high-risk category.
- **Evaluate the model** using metrics such as **accuracy**, **precision**, **recall**, **F1-score**, and the **AUC-ROC curve**.
- Provide **actionable insights** for banks to improve their risk management strategies, such as adjusting credit policies or targeting clients for financial counseling.

The goal is not just to build a predictive model but to also enable banks to take **proactive measures** to identify high-risk clients early in the process, thereby minimizing financial risk and optimizing the bank's resources.

1.5 Structure of the Project

The structure of this book is designed to provide a clear, step-by-step guide to the project, starting from the **theoretical foundation** to the final model implementation and evaluation. The chapters are organized as follows:

- **Chapter 1: Introduction & Project Overview**
  Introduces the project, the problem statement, the dataset, and the project's objectives. Also provides a brief introduction to the **theoretical foundation** of the Logistic Regression model.
- **Chapter 2: Understanding Logistic Regression**
  Explains the **theoretical foundation** behind **Logistic Regression**. It covers the mathematical formulation of the model, its assumptions, and its application to binary classification problems.
- **Chapter 3: Exploratory Data Analysis (EDA)**
  Discusses the **Exploratory Data Analysis (EDA)** phase of the project, where the dataset is analyzed through **visualization** and **statistical summaries**. This chapter will identify key patterns, trends, and relationships in the data that will guide feature selection for the model.

- **Chapter 4: Power BI Integration and Visualization** Focuses on the **Power BI** tool for modeling and visualizing the data and the relationships between variables. It provides an interactive dashboard and data visualizations that help in understanding the dataset. The insights from **Power BI** are independent of the Logistic Regression model but will complement the findings from the **EDA** and help visualize the final model's results.
- **Chapter 5: Logistic Regression Model Implementation** Covers the step-by-step **implementation** of the **Logistic Regression model**, including, model fitting, evaluation metrics, and performance analysis.

1.6 Theoretical Foundation

Logistic Regression is a statistical method used to model binary outcomes (e.g., **high risk** or **low risk**) based on one or more predictor variables. It is a type of **generalized linear model** that predicts the probability of a binary response by using a **logistic function** (also known as the **sigmoid function**).

The logistic function maps any real-valued number into a probability between 0 and 1:

$$P(Y = 1 \mid X) = 1/1 + e - (\beta 0 + \beta 1X1 + \beta 2X2 + \ldots + \beta nXn)$$

Where:

- $P(Y = 1 \mid X)$ is the probability that the target variable Y (e.g., **Risk**) is 1 (high-risk).
- $X1, X2, \ldots, Xn$ are the predictor variables (e.g., **age**, **income**, **credit card balance**, etc.).
- $\beta 0, \beta 1, \ldots, \beta n$ are the coefficients that represent the influence of each feature on the log-odds of the target variable.

Logistic Regression is particularly well-suited for classification problems because it provides a clear probability that can be used to assign clients to either the high-risk or low-risk class. The model is trained by **maximum likelihood estimation** (MLE), which seeks to find the best-fitting parameters $\beta 0, \beta 1, \ldots, \beta n$ that maximize the likelihood of the observed data.

Chapter 2

Theoretical Foundation of Logistic Regression

2.1 Introduction to Binary Classification

In many real-world problems, the target variable (also known as the dependent variable) is categorical, often with two possible outcomes. For example:

- **High-risk vs. low-risk clients** in banking,
- **Yes vs. No** decisions (e.g., will a customer buy a product?),
- **Positive vs. negative disease diagnosis** in healthcare.

These are examples of **binary classification problems**, where the goal is to predict which of two categories an observation belongs to. Logistic regression is a powerful method for solving binary classification problems.

2.2 The Logistic Regression Model

Logistic regression is a type of regression analysis used for predicting the outcome of a binary dependent variable. Unlike linear regression, which predicts continuous values, logistic regression predicts probabilities that lie between 0 and 1, making it suitable for binary classification.

Logistic Function (Sigmoid Function)

The **logistic function**, also called the **sigmoid function**, is the core of logistic regression. It transforms any real-valued number into a value between 0 and 1, suitable for probability estimation. The function is defined as:

$$P(Y = 1 \mid X) = 1/1 + e\text{^} - (\beta 0 + \beta 1X1 + \beta 2X2 + \cdots + \beta nXn)$$

Where:

- $P(Y = 1 \mid X)$ is the probability of the target variable being 1 (e.g., high risk).
- $X1, X2, \ldots, Xn$ are the features (predictors).
- $\beta 0, \beta 1, \ldots, \beta n$ are the model coefficients.

The sigmoid function maps the linear combination of predictors to a probability. The output is interpreted as the probability of an observation belonging to class 1 (e.g.,

high-risk), while 1−P(Y=1|X) is the probability of it belonging to class 0 (e.g., low-risk).

Odds and Log-Odds

In logistic regression, we often refer to the **odds** and **log-odds** of an event:

- **Odds**: The ratio of the probability of the event occurring to the probability of the event not occurring:

$$Odds = P(Y = 1 \mid X)/1 - P(Y = 1 \mid X)$$

- **Log-Odds**: The natural logarithm of the odds:

$$log\ (P(Y = 1 \mid X)/1 - P(Y = 1 \mid X))$$

Logistic regression models the log-odds of the target variable as a linear combination of the predictor variables.

Odds Ratio

Exponentiating the coefficients of the logistic regression model yields the **odds ratio (OR)**, which tells us how the odds of the event change with a one-unit increase in the predictor variable:

$$OR = e\beta n$$

For instance, if $\beta 1 = 0.5$, then e^0.5 ≈ 1.65, meaning that for every one-unit increase in the corresponding feature, the odds of being classified as high-risk increase by 65%.

2.3 Comparison with Linear Regression

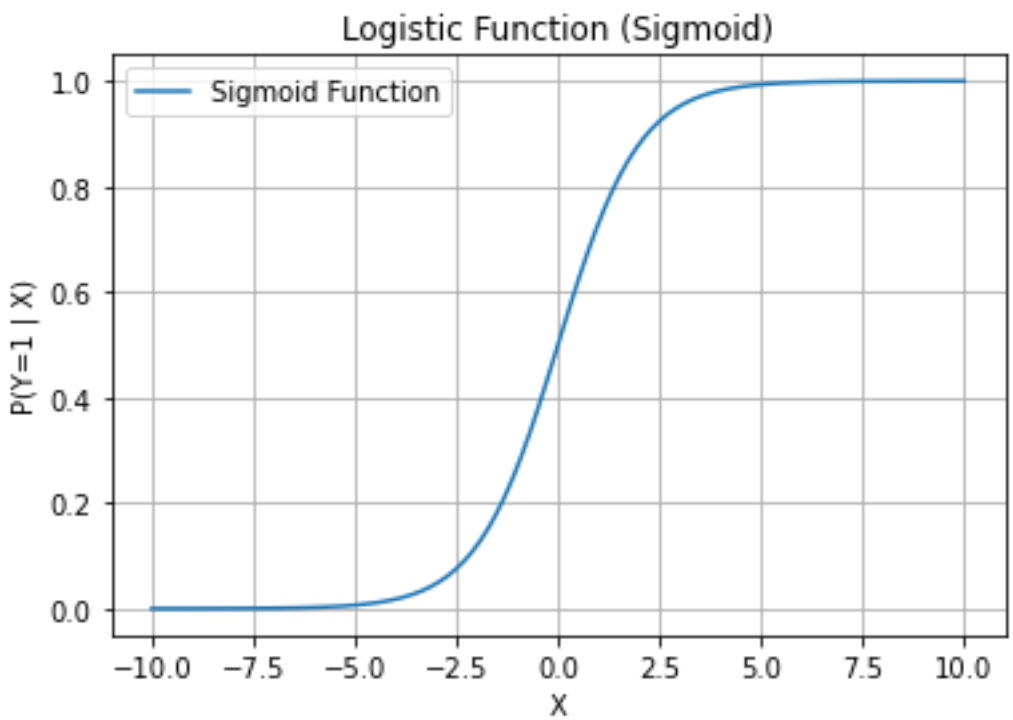
Aspect	Linear Regression	Logistic Regression
Type of Problem	Predicts continuous outcomes	Predicts binary outcomes (classification)
Output	Continuous values (real numbers)	Probabilities (values between 0 and 1)
Target Variable	Can take any real value	Categorical, binary (0 or 1)
Function	Linear function $Y = \beta 0 + \beta 1X1 + \cdots + \beta nXnY$	Logistic (sigmoid) function ($P(Y = 1)$)
Assumptions	Linear relationship between predictors and target	Log-linear relationship between predictors and target
Error Term	Normally distributed errors	Assumes no error term but focuses on the likelihood of a binary outcome
Evaluation Metrics	Mean Squared Error (MSE), R-squared	Accuracy, Precision, Recall, F1-Score, Log-Loss
Example	Predicting house prices	Predicting whether a customer will default on a loan

Key Differences:

- **Linear Regression** is used for continuous outcomes, while **Logistic Regression** is used for binary outcomes.
- **Linear Regression** assumes that the relationship between the predictors and target is linear, whereas **Logistic Regression** models the log-odds of the outcome, which is more appropriate for categorical data.
- The output of linear regression is a continuous number, while logistic regression outputs a probability between 0 and 1.

Visualizing the Logistic Function

A plot of the logistic function shows how the probability increases smoothly from 0 to 1 as the input values increase. The curve of the sigmoid function looks like an "S" shape:



This curve shows the output probability of logistic regression. As the input increases, the probability approaches 1, while as the input decreases, it approaches 0.

2.4 Assumptions of Logistic Regression

Before applying logistic regression, we should ensure that the following assumptions are met:

1. **Binary Outcome**: The target variable must be binary (e.g., 0 or 1, high-risk or low-risk).
2. **Independence of Observations**: The observations should be independent of each other. This assumption can be violated in time-series or clustered data.
3. **Linearity of Log-Odds**: There should be a linear relationship between the log-odds of the dependent variable and the independent variables. This can be tested using plots of the predictor variables versus the log-odds.

4. **No Multicollinearity**: Predictor variables should not be highly correlated with one another. Multicollinearity can destabilize the estimation of coefficients.
5. **Large Sample Size**: Logistic regression generally requires a sufficiently large sample size to produce reliable coefficient estimates.

2.5 Model Evaluation Metrics

To assess the performance of a logistic regression model, the following metrics are commonly used:

- **Accuracy**: The proportion of correct predictions out of all predictions made. It is calculated as:

$$Accuracy = TP + TN/TP + TN + FP + FN$$

- **Precision**: The proportion of positive predictions that are actually correct:

$$Precision = TP/TP + FP$$

- **Recall (Sensitivity)**: The proportion of actual positives that are correctly identified by the model:

$$Recall = TP/TP + FN$$

- **F1-Score**: The harmonic mean of precision and recall:

$$F1 = 2 \times Precision \times Recall/Precision + Recall$$

- **Confusion Matrix**: A table that summarizes the performance of the classification model. It shows the number of true positives, true negatives, false positives, and false negatives.

2.6 Applications of Logistic Regression

Logistic regression is used in many domains, including:

- **Healthcare**: Predicting the likelihood of a patient developing a certain disease.
- **Marketing**: Classifying whether a customer will purchase a product or not.

- **Banking**: Predicting whether a client will default on a loan or if a customer is high-risk or low-risk.

2.7 Conclusion

Logistic regression is a powerful tool for binary classification problems. Its ability to provide probabilities, interpret coefficients, and work well with imbalanced datasets makes it an ideal choice for many practical applications. In this chapter, we have explored its mathematical foundations, assumptions, and how it can be applied to solve real-world problems.

Chapter 3

Exploratory Data Analysis (EDA)

3.1 Introduction to Exploratory Data Analysis

Exploratory Data Analysis (EDA) is a critical first step in any data science project. It involves visually and statistically summarizing a dataset's main characteristics before applying more advanced machine learning models. In this chapter, we perform EDA on a dataset containing information about clients, including demographic features, financial behaviors, and risk classifications.

The main objectives of EDA are:

- **Understanding the structure of the dataset**
- **Identifying relationships** between different variables
- **Spotting trends** and **outliers**
- **Preparing data** for modeling

Before proceeding to model training, we must ensure that the dataset is properly cleaned, free of missing values or duplicates, and formatted for analysis. In this chapter, we focus on the initial steps of data cleaning, feature analysis, and visualization to uncover insights about the clients' financial behaviors.

3.2 Data Cleaning and Preprocessing

3.2.1 Missing Data

The dataset was inspected for **missing values** to ensure the integrity of our analysis. Upon checking, it was found that **no missing values** were present in any of the columns. This is a crucial step as missing data can lead to biased results, especially in statistical models. In practice, missing data is often imputed or removed, but since our dataset is complete, no further action was needed.

```python
# Check for missing values
missing_values = df.isnull().sum()
print("Missing values per column:\n", missing_values)
```

```
Missing values per column:
 Client ID                   0
Name                        0
Age                         0
Location ID                 0
Joined Bank                 0
Banking Contact             0
Nationality                 0
Occupation                  0
Fee Structure               0
Loyalty Classification      0
Estimated Income            0
Superannuation Savings      0
Amount of Credit Cards      0
Credit Card Balance         0
Bank Loans                  0
Bank Deposits               0
Checking Accounts           0
Saving Accounts             0
Foreign Currency Account    0
Business Lending            0
Properties Owned            0
Risk Weighting              0
BRId                        0
GenderId                    0
IAId                        0
Income Band                 0
dtype: int64
```

3.2.2 Duplicate Data

Duplicate entries can skew analysis and affect the validity of statistical conclusions. The dataset was checked for duplicate rows, and it was confirmed that **no duplicates** existed. This ensures that each client's data is unique and that our analysis remains accurate.

```python
# Checking for duplicate rows in the dataset
duplicate_rows = df.duplicated().sum()

# Printing the number of duplicate rows
print(f"Number of duplicate rows: {duplicate_rows}")
```

```
Number of duplicate rows: 0
```

3.2.3 Data Type Verification

Data types for each column were verified to ensure proper handling. For example, the **'Joined Bank'** column was initially in an object format and was converted to the **datetime format**. This ensures that time-based analyses and operations can be carried out correctly.

```python
# Converting 'Joined Bank' to datetime
df['Joined Bank'] = pd.to_datetime(df['Joined Bank'], dayfirst=True)
```

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 25 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Client ID                  3000 non-null   object
 1   Name                       3000 non-null   object
 2   Age                        3000 non-null   int64
 3   Location ID                3000 non-null   int64
 4   Joined Bank                3000 non-null   datetime64[ns]
 5   Banking Contact            3000 non-null   object
 6   Nationality                3000 non-null   object
 7   Occupation                 3000 non-null   object
 8   Fee Structure              3000 non-null   object
 9   Loyalty Classification     3000 non-null   object
 10  Estimated Income           3000 non-null   float64
 11  Superannuation Savings     3000 non-null   float64
 12  Amount of Credit Cards     3000 non-null   int64
 13  Credit Card Balance        3000 non-null   float64
 14  Bank Loans                 3000 non-null   float64
 15  Bank Deposits              3000 non-null   float64
 16  Checking Accounts          3000 non-null   float64
 17  Saving Accounts            3000 non-null   float64
 18  Foreign Currency Account   3000 non-null   float64
 19  Business Lending           3000 non-null   float64
 20  Properties Owned           3000 non-null   int64
 21  Risk Weighting             3000 non-null   int64
 22  BRId                       3000 non-null   int64
 23  GenderId                   3000 non-null   int64
 24  IAId                       3000 non-null   int64
dtypes: datetime64[ns](1), float64(9), int64(8), object(7)
memory usage: 586.1+ KB
```

```python
# Checking the data types of each column
data_types = df.dtypes  # Ensure you're using df_csv as the DataFrame name

# Displaying the data types of the first few columns
print("Data Types of the first few columns:")
print(data_types.head())
```

```
Data Types of the first few columns:
Client ID      object
Name           object
Age             int64
Location ID     int64
Joined Bank    object
dtype: object
```

3.3 Visualizing the Data and Gaining Insights

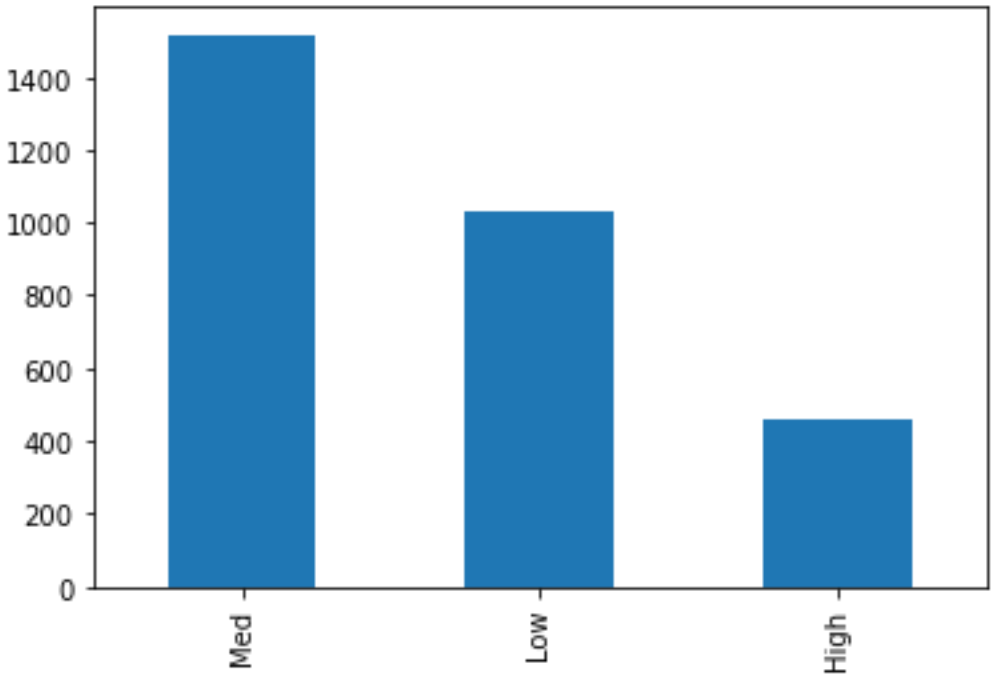
After performing initial data cleaning and preprocessing, we explored the dataset visually to uncover any underlying patterns and trends. This phase is crucial as it allows us to detect potential relationships between variables, identify outliers, and provide insight into the financial behaviors of clients. Below are the key visualizations and insights from the data exploration process.

3.3.1 Distribution of Income Band

```python
bins = [0, 100000, 300000, float('inf')]
labels = ['Low', 'Med', 'High']

df['Income Band'] = pd.cut(df['Estimated Income'], bins=bins, labels=labels, right=False)
```

We first segmented the **Estimated Income** into three distinct categories—**Low**, **Medium**, and **High**. The distribution of clients across these categories revealed the following:

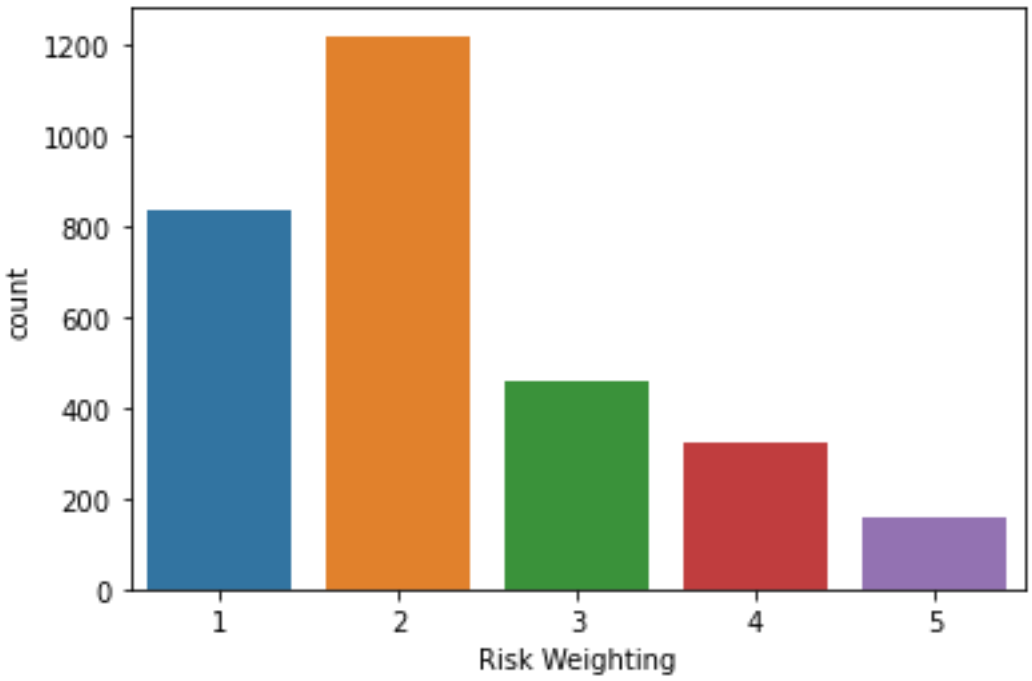


- The **Medium Income** band contained the majority of the clients, followed by the **Low Income** band, and finally the **High Income** band.

Insight: The **Medium Income** group makes up the largest portion of the dataset, indicating that the majority of clients fall within this income range. This segmentation could have implications for financial planning and risk assessment, especially in determining how financial institutions might approach different income groups.

3.3.2 Risk Weighting Distribution

Next, we examined the **Risk Weighting** variable, which categorizes clients into different risk groups. The distribution showed that:

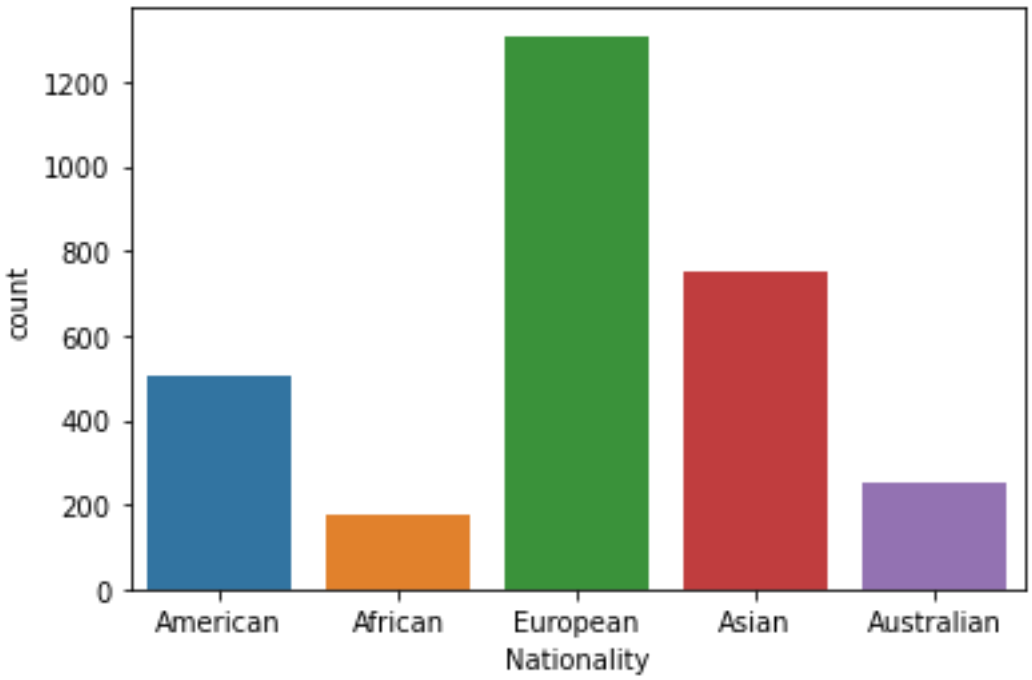


- **Risk Weighting 2** (moderate risk) was the most common classification, followed by **Risk Weighting 1** (low risk). The dataset had fewer clients in the higher-risk categories (3, 4, and 5).

Insight: The dominance of clients classified under **Risk Weighting 2** suggests that the financial institution in this dataset faces a larger proportion of moderately risky clients. This finding is crucial for model training, as the focus might be on improving the risk classification of this group.

3.3.3 Nationality Distribution

When we examined the **Nationality** variable, we saw a marked difference in the number of clients by nationality. The following trends were observed:

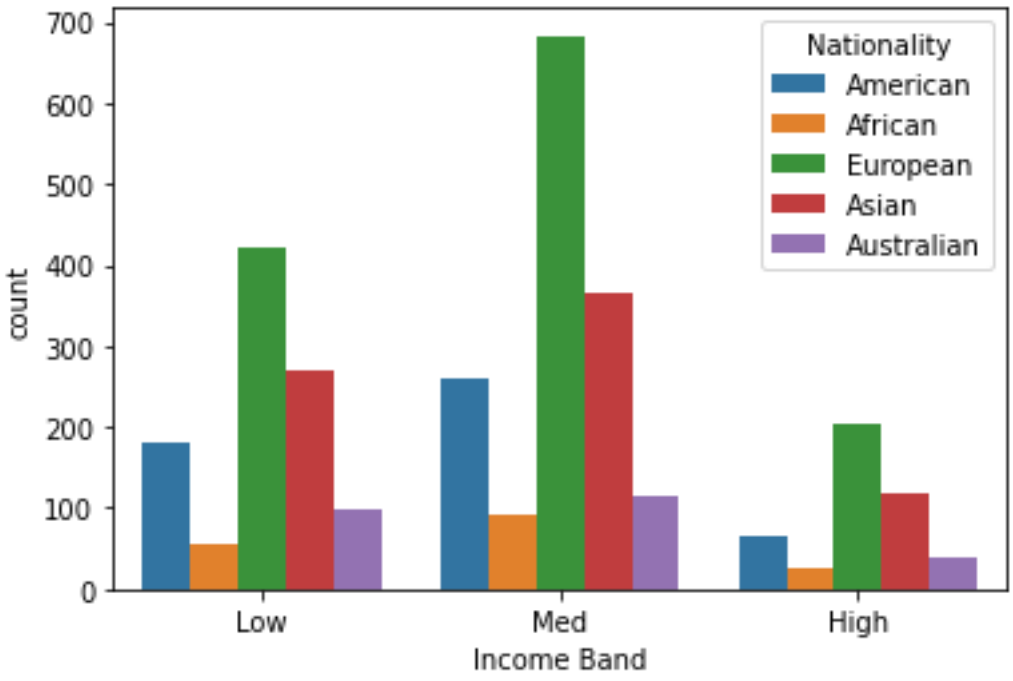


- **European** nationality had the largest number of clients, followed by **American** and **Asian** nationalities. **African** and **Australian** nationalities represented smaller portions of the dataset.

Insight: The high representation of **European** clients in the dataset may indicate that the financial institution primarily serves clients from European backgrounds. Understanding nationality-based financial behaviors could be essential in refining the risk assessment model to better serve clients from different geographical backgrounds.

3.3.4 Income Band by Nationality

The visualization of **Income Band** distribution across different **Nationalities** revealed some interesting insights:



- **European clients** were more likely to fall within the **Medium Income** band.
- **American clients** were more evenly distributed across **Low** and **Medium Income** bands.
- **Asian clients** appeared to be more concentrated in the **Low Income** band.

Insight: This breakdown suggests that **European clients** may generally have higher financial stability, while **Asian clients** are disproportionately represented in the **Low Income** group. This differentiation could be useful in predictive modeling when assessing risk based on nationality and income.

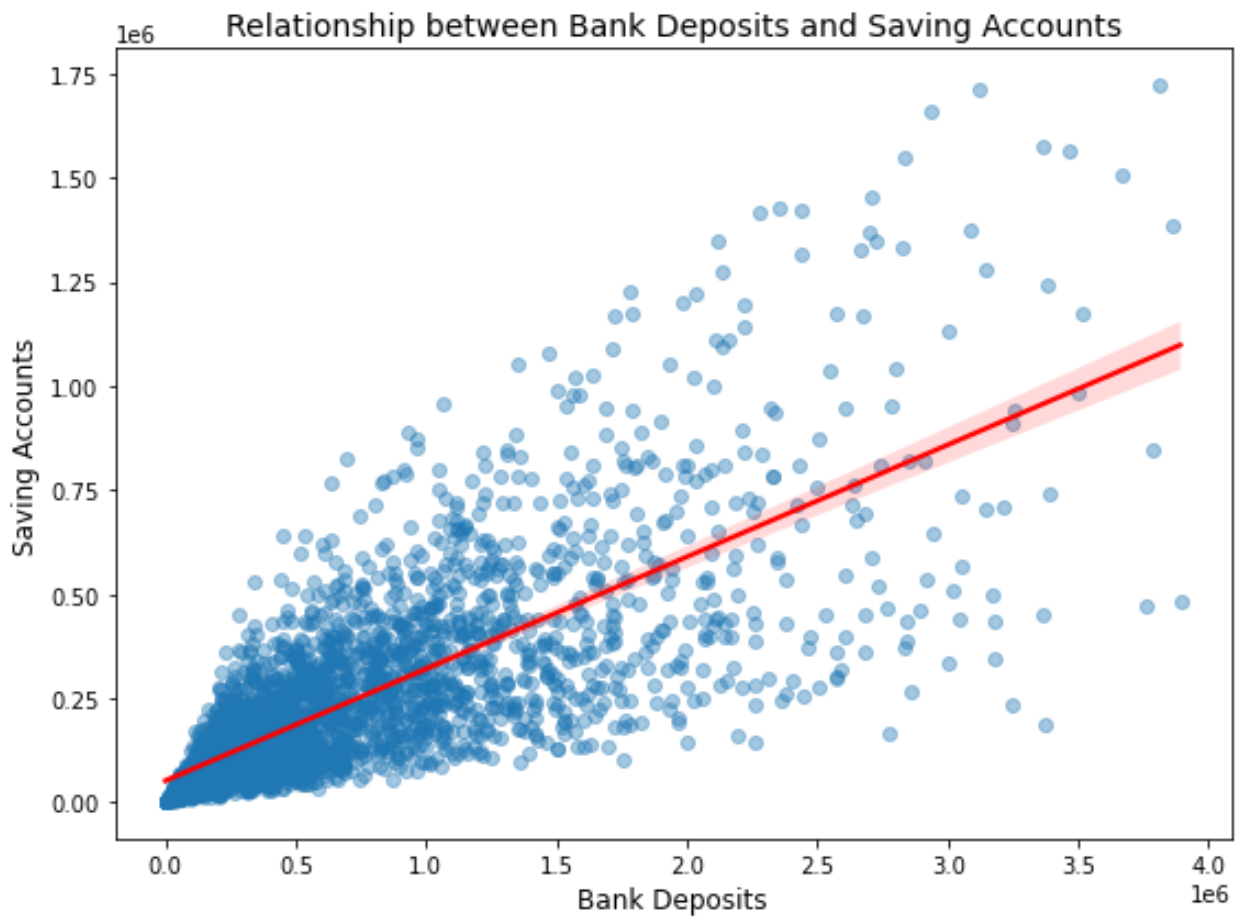
3.3.5 Correlation Matrix

The **correlation matrix** for numerical features indicated several notable relationships:

- **Bank Deposits** and **Saving Accounts** exhibited a strong positive correlation (0.75), meaning clients with larger bank deposits tend to also have higher savings accounts.
- **Checking Accounts** showed a significant positive correlation with **Bank Deposits** (0.84), which suggests that clients with checking accounts tend to also maintain larger bank deposits.

Insight: The correlation between **Bank Deposits** and **Saving Accounts** underscores the importance of these features in assessing a client's financial capacity. Similarly, the correlation between **Bank Loans** and **Bank Deposits** could inform how clients' borrowing behaviors are tied to their deposit holdings.

3.3.6 Scatter Plot between Bank Deposits and Saving Accounts



The scatter plot of **Bank Deposits** against **Saving Accounts** revealed a positive linear relationship, with an upward trend suggesting that as **Bank Deposits** increase, **Saving Accounts** also tend to increase. This observation aligns with the earlier correlation matrix, reinforcing the idea that clients with higher deposits are likely to have greater savings.

Insight: This positive correlation suggests that **Bank Deposits** and **Saving Accounts** are closely related and may provide valuable predictive features for determining client risk. Understanding this relationship is key for feature engineering in machine learning models.

3.4 Conclusion of EDA

In this chapter, we explored the dataset through visualizations to uncover key insights about the financial behaviors of the clients. The analysis revealed several important trends:

- **Income Band Distribution**: The **Medium Income** group is dominant, while **High Income** clients are fewer.
- **Risk Weighting**: **Risk Weighting 2** was the most common classification, highlighting the large number of moderately risky clients.
- **Nationality**: **European clients** dominated the dataset, which may suggest specific financial behaviors associated with this group.
- **Financial Relationships**: There were strong correlations between **Bank Deposits** and **Saving Accounts**, as well as moderate correlations between **Bank Loans** and **Bank Deposits**, which can serve as useful features in predictive models.

These insights will inform the next steps in the modeling process, where we will leverage these features to build a logistic regression model for predicting risk.

Chapter 4:

Power BI Modeling and Visualization

4.1 Introduction to Power BI

4.1.1 What is Power BI?

Power BI is a powerful and versatile business analytics tool developed by Microsoft that allows users to visualize and share insights from their data. It plays a crucial role in transforming raw, often complex, datasets into accessible and actionable visualizations, which can be used to drive informed business decisions.

At its core, Power BI allows users to connect to various data sources, clean and transform data, model relationships, and then create interactive dashboards and reports. It enables users to quickly turn large amounts of data into easy-to-understand visual insights, making it an indispensable tool in business intelligence (BI).

Power BI is composed of several components:

- **Power BI Desktop**: A desktop application used for designing reports and data models. It is the primary tool used by data analysts to connect to data sources, clean data, and design reports.
- **Power BI Service**: A cloud-based service that allows users to share and collaborate on reports and dashboards.
- **Power BI Mobile**: A mobile application that allows users to access their reports and dashboards on smartphones and tablets.

4.1.2 Why Power BI is Important for Data Analysis

Power BI provides several key benefits that make it invaluable for data analysis and business intelligence:

- **Real-time Data Insights**: Power BI enables users to create real-time dashboards, offering up-to-date insights that can be accessed by stakeholders instantly. This capability is essential in today's fast-paced business environment.

- **Interactive Visualizations**: Power BI's ability to present data interactively allows users to explore data through visual elements such as bar charts, pie charts, line graphs, and maps. Users can click on elements in the visuals to filter and drill down into the data, uncovering deeper insights.
- **Data Integration**: Power BI can connect to a wide variety of data sources, ranging from local files (e.g., Excel, CSV) to cloud-based platforms (e.g., Azure, Google Analytics) and databases (e.g., SQL Server). This flexibility makes it easier to combine and analyze data from multiple sources.
- **Collaborative Capabilities**: With Power BI Service, teams can collaborate, share reports, and communicate insights, fostering better decision-making and teamwork.

4.1.3 Key Features of Power BI

There are several key features of Power BI that enhance its data analysis and visualization capabilities:

- **Data Modeling**: Power BI allows users to create relationships between different tables of data, allowing them to easily combine data from multiple sources into a single, cohesive dataset.
- **DAX (Data Analysis Expressions)**: DAX is a formula language used in Power BI to create custom calculations, aggregations, and key performance indicators (KPIs) directly within the data model. DAX enables users to perform more advanced calculations that go beyond basic summing and averaging.
- **Visualizations**: Power BI offers a wide range of visualization types, such as bar charts, line charts, scatter plots, maps, and tables. These visualizations can be customized to present the data in the most intuitive way for the target audience.
- **Power Query**: Power Query is a tool within Power BI used for data cleaning and transformation. It allows users to reshape, filter, and transform data before it is loaded into the data model.

4.2 Introduction to Data Modeling in Power BI

4.2.1 What is Data Modeling in Power BI?

Data modeling in Power BI refers to the process of organizing and structuring data to enable easy and efficient analysis. This process includes defining relationships between tables, calculating key metrics, and transforming raw data into usable insights. The goal of data modeling is to provide a framework where data can be analyzed dynamically using interactive visualizations.

The core of Power BI data modeling lies in creating **relationships** between tables, defining **measures** and **calculated columns**, and setting the **granularity** of the data. These components work together to ensure that the data model is both comprehensive and efficient for data analysis.

4.2.2 Fact Tables and Dimension Tables

In Power BI, as in most data modeling techniques, data is typically organized into two main types of tables:

- **Fact Tables**: These contain quantitative data, usually numerical values that you want to analyze, such as **sales**, **expenses**, or **loan amounts**. Fact tables are typically linked to multiple dimension tables to give context to the numbers. In the context of the banking dataset, for example, the **Transactions** table or **Loans** table could be considered a fact table, as it contains data that can be aggregated or summarized.
- **Dimension Tables**: These provide the context for the fact data. Dimension tables contain descriptive attributes related to the facts, such as **client demographics**, **product types**, or **time periods**. In the banking dataset, tables like **Clients**, **Accounts**, or **Loan Types** could be dimension tables, as they provide more details about the data in the fact tables.

A **star schema** or **snowflake schema** is often used to model relationships between fact and dimension tables, and these schemas help structure the model for efficient querying.

1. **Star Schema**: In the star schema, a central fact table (e.g., transactions or loans) is connected to multiple dimension tables (e.g., client demographics, product types). This schema is simple, as each dimension table is directly linked to the fact table.

2. **Snowflake Schema**: In this schema, dimension tables are normalized into additional sub-dimension tables, creating a more complex structure. This may be used if you want to reduce redundancy in dimension data or have a more granular level of detail.

4.2.3 Granularity in Data Modeling

Granularity refers to the level of detail in the dataset. Choosing the right granularity is crucial in data modeling because it determines the type of analysis you can perform. There are two main types of granularity:

- **Transaction-Level Granularity**: Data is captured at the individual transaction or event level. For instance, each row in the table represents a single transaction made by a client.
- **Client-Level Granularity**: Data is aggregated at the client level, where each row represents an individual client with aggregated information (e.g., total balance, total loans, etc.).

The **granularity** of the data model affects how tables are related and the types of calculations that can be performed. For example, if you're working with client-level granularity, the model would allow you to analyze data related to clients' total loan amounts or income, without breaking it down into each transaction.

4.2.4 Relationships in Power BI

In Power BI, relationships are defined between tables based on common columns (such as **Client ID**, **Product ID**, etc.). These relationships help Power BI link data from different tables and provide a comprehensive view for analysis. Relationships can be:

- **One-to-One**: Each row in one table corresponds to one row in another table (e.g., a client ID in the client table corresponding to a unique entry in the client details table).
- **One-to-Many**: One record in a dimension table (like Clients) corresponds to multiple records in a fact table (like Transactions). For instance, one client can have many transactions.

- **Many-to-Many**: This type of relationship is more complex and requires special handling in Power BI. It occurs when multiple records in both tables are related to each other (e.g., clients and products, where each client can have multiple products, and each product can belong to multiple clients).

4.2.5 Key Data Modeling Concepts in Power BI

1. **DAX Measures and Calculations**: DAX (Data Analysis Expressions) is used in Power BI to create calculated columns and measures that allow for more sophisticated data analysis. For example, we can use DAX to calculate **Total Loans** for a client or **Risk Scores** based on specific criteria. These calculated metrics are essential for building KPIs and making data-driven decisions.
2. **Calculated Columns vs. Measures**:
    - **Calculated Columns**: These are columns that are calculated row by row based on the existing data. They become part of the data model and are useful for creating new attributes or categories in your data (e.g., categorizing clients based on their total deposits).
    - **Measures**: Measures are dynamic calculations that are performed when a report is viewed or interacted with. Measures respond to filters and are useful for aggregating data, such as summing, averaging, or counting values across different segments (e.g., total loan amount or average income).

4.3 The Data Model for Our Dataset

In this section, we will explore the data model implemented in Power BI, focusing on the relationships between the tables, the granularity of the data, and how these elements come together to create a coherent structure for data analysis. The goal of the data model is to enable efficient exploration and analysis of the bank's client data, financial behaviors, and risk classifications.

4.3.1 Overview of the Dataset

The dataset used in this project encompasses various aspects of client information, financial activities, and banking relationships. It is structured into multiple tables, with each table focusing on a specific domain. By combining

these tables, we can uncover insights about the clients' financial behaviors, assess their risk profiles, and observe trends across different client demographics.

The key components of the dataset include:

- **Client Information**: This contains demographic details about each client, including age, income, gender, and nationality. This information is critical for segmenting clients based on their financial profiles and behaviors.
- **Account and Transaction Data**: This includes records of client banking accounts, such as **bank deposits**, **savings accounts**, **checking accounts**, and more. The transactions associated with each account, including deposits and withdrawals, provide a comprehensive view of a client's financial activity.
- **Risk Assessment**: The dataset includes a **Risk Weighting** variable, which classifies clients based on their creditworthiness. The risk categories (e.g., low, medium, high) help the bank assess and manage its financial exposure to each client.

4.3.2 Modeling the Data in Power BI

The first step in creating the Power BI data model was importing the various data tables from the dataset. Each table corresponds to a different aspect of the bank's operations, such as clients, financial products, and relationships. After importing the data, we needed to establish relationships between these tables so that we could perform cross-table analysis.

In Power BI, relationships are established based on primary keys in one table that correspond to foreign keys in another. For example, the Clients table contains a Client ID field, which serves as the primary key that can be related to other tables, such as the Accounts and Transactions tables, which contain a Client ID as a foreign key.

The goal of the data modeling process is to ensure that users can interact with the data in a meaningful way—by enabling them to filter, slice, and aggregate data from different tables with ease. Power BI's star schema structure allows for seamless querying, enabling visualizations and reports that provide clear insights into the bank's client base and operations.

4.3.3 Relationships in the Data Model

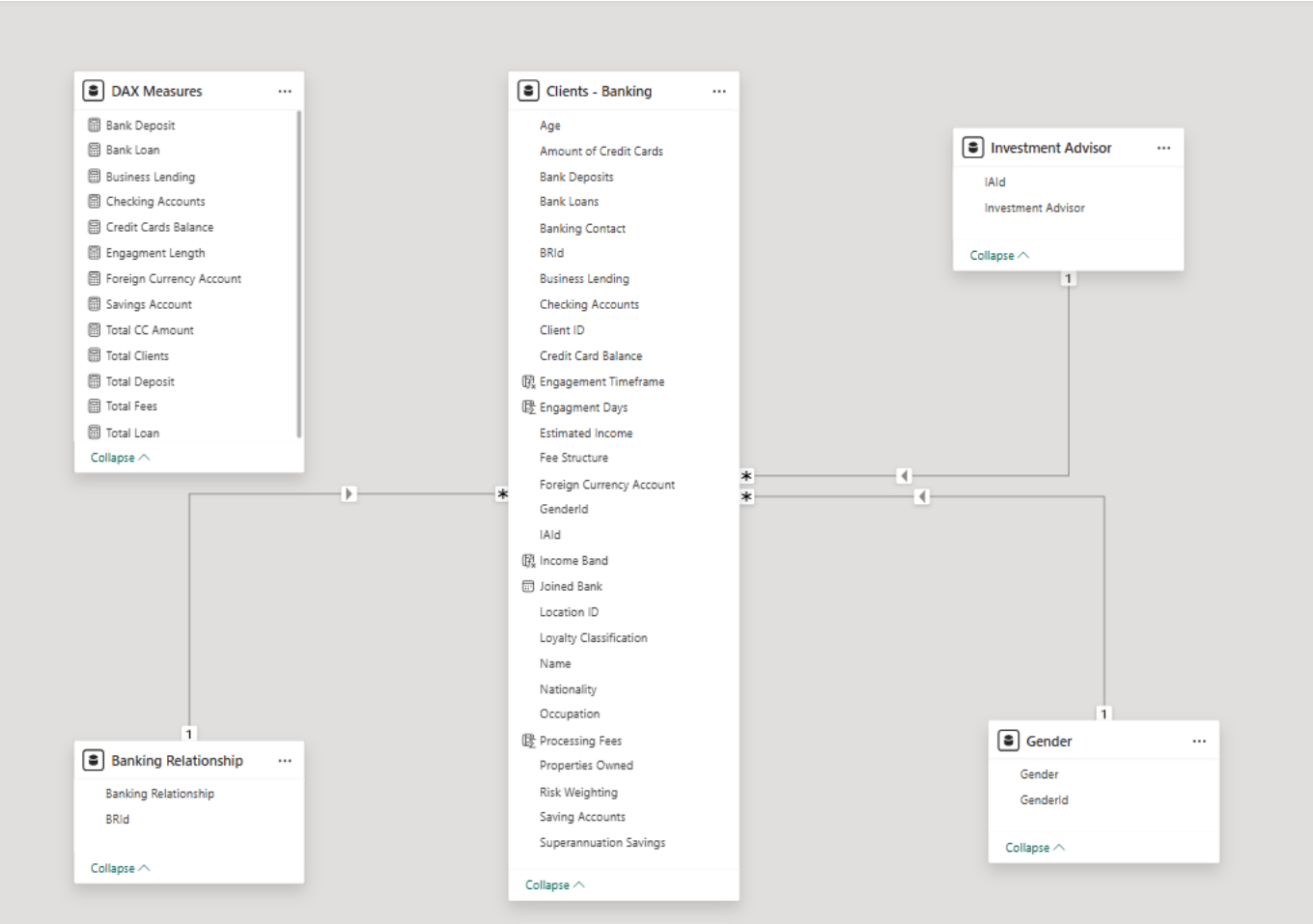
In the data model we constructed in Power BI, we have **one fact table** and **several dimension tables**. Let's first discuss what **fact** and **dimension tables** are and then move on to the relationships between them:

- **Fact Table**: This is where the **measurable data** is stored. The fact table typically contains the main data of interest, such as sales, profits, or in our case, **banking-related financial activities**. It is the core table where data is aggregated and calculations are made.
- **Dimension Tables**: These are the tables that contain the descriptive attributes (or metadata) used to provide context to the fact data. For instance, client information such as **gender**, **investment advisor**, and **banking relationship** are stored in dimension tables.

Key Tables in Our Data Model

1. **Clients - Banking (Fact Table)**:
    - This table is the **fact table** of our model. It contains the main data related to each client's banking activities. The **fact** here includes metrics and financial information such as **Bank Deposits**, **Loans**, **Credit Card Balances**, **Savings Accounts**, **Foreign Currency Accounts**, and **Estimated Income**.
    - **Client ID** serves as the key field, linking this table to other related dimension tables, allowing us to perform detailed analysis on the banking data.
2. **Banking Relationship (Dimension Table)**:
    - This table provides descriptive data about the banking relationships of each client. It includes a **Banking Relationship ID (BRId)**, which connects the client's banking relationships to specific accounts and services.
    - It helps describe the context of the client's engagement with the bank.
3. **Investment Advisor (Dimension Table)**:
    - The **Investment Advisor** dimension table links each client to an **Investment Advisor ID (IAd)**, which identifies which advisor has interacted with the client. This allows for analysis based on the clients' relationship with their investment advisor.
4. **Gender (Dimension Table)**:
    - The **Gender** dimension table categorizes clients based on their gender. It contains **Gender** and **GenderID**, which provide a categorical

breakdown for analyzing how gender correlates with other client behaviors and banking activities.



Relationships Between the Tables

The relationships between these tables are essential to create a unified view of the data in Power BI. These relationships allow for data aggregation, filtering, and analysis across different tables:

1. **Clients - Banking** to **Banking Relationship**:
    - The **Clients - Banking** table is linked to the **Banking Relationship** table via **Client ID**, which ensures that each banking activity (e.g., deposits, loans) is associated with a specific client's banking relationship.
2. **Clients - Banking** to **Investment Advisor**:

- The **Clients - Banking** table is connected to the **Investment Advisor** table through the **Investment Advisor ID (IAd)**. This relationship helps link client data with the investment advisors they interact with.

3. **Clients - Banking** to **Gender**:
   - The **Clients - Banking** table is also connected to the **Gender** table through **Client ID**, providing a way to analyze the data based on gender. This relationship allows for segmentation and comparison of financial behaviors across gender groups.

The relationships in the data model are **one-to-many** in nature, where each client (from the **Clients - Banking** table) can have multiple relationships, advisors, or gender attributes. However, the **Banking Relationship**, **Investment Advisor**, and **Gender** tables represent individual categories or attributes related to each client.

4.3.4 Granularity of the Data

Granularity refers to the level of detail captured in the dataset. In this model, we observe two levels of granularity:

1. **Client-Level Granularity**: The **Clients Table** holds data at the **client level**, where each row represents an individual client. This table includes demographic and financial data about each client, aggregated at the client level (e.g., total loans, total deposits, income).

2. **Transaction-Level Granularity**: The **Transactions Table** holds data at the **transaction level**, where each row represents an individual transaction made by a client. It captures details such as transaction amount, date, and type. This table allows for more granular analysis of client behaviors and transaction trends.

4.3.5 Measures and DAX Calculations

What is DAX (Data Analysis Expressions)?

DAX (Data Analysis Expressions) is a powerful formula language used in Power BI, Power Pivot, and SQL Server Analysis Services (SSAS) for data modeling and analysis. It enables users to create **custom calculations** and **aggregations** that are

essential for building insightful visualizations, making it a core component of Power BI's data transformation and reporting capabilities.

DAX is similar to Excel formulas but goes beyond simple calculations. It is designed to work with data in a columnar format, allowing users to define **measures**, **calculated columns**, and **calculated tables** that provide a dynamic and insightful view of the data.

In Power BI, DAX allows us to calculate values based on the relationships within the data model. It helps in the creation of dynamic reports where the calculations change depending on filters, slicers, and other user interactions with the report.

Types of DAX Formulas

In Power BI, DAX formulas can be classified into three main types:

1. **Measures**:
   - Measures are calculations that **aggregate data** based on a formula. These are dynamic calculations that change depending on the context of the report or visual (e.g., filtering or slicing). Measures are **not stored in the data model**, and they are computed only when needed in the report.
2. **Calculated Columns**:
   - Calculated columns are similar to measures, but they are **stored in the data model** as part of the table. They perform row-wise calculations for each entry in the dataset. Once the column is created, it is available for use just like any other column in the table.
3. **Calculated Tables**:
   - Calculated tables allow users to define entire tables based on DAX formulas. These tables are **created dynamically** in the data model, based on calculations defined by the user, and can be used like any other table in the report.

The final piece of the puzzle is the creation of DAX measures to aggregate data and calculate key performance indicators (KPIs). These measures allow for dynamic calculations when filters are applied, making it easier to derive insights without manually performing calculations.

Some of the key DAX measures in our Power BI model include:

- **Total Deposits**: The sum of all deposits made by clients. This measure aggregates data from the **Accounts** and **Transactions** tables to show the total deposits for all clients.

```
Total Deposit = [Bank Deposit] + [Savings Account] + [Foreign Currency Account] + [Checking Accounts]
```

- **Total Loans**: The total sum of loans issued to clients, calculated from the **Loans Table**.

```
Total Loan = [Bank Loan] + [Business Lending] + [Credit Cards Balance]
```

- **Total Fees**: The sum of all fees charged to clients. This measure can be calculated from relevant transaction or fee data.

```
Total Fees = SUMX('Clients - Banking' , [Total Loan] * 'Clients - Banking'[Processing Fees] )
```

- **Total Clients**: The total number of clients in the dataset. This measure counts the rows in the **Clients Table**.

```
Total Clients = DISTINCTCOUNT('Clients - Banking'[Client ID] )
```

These measures are calculated dynamically by Power BI and are used to populate visualizations. The ability to calculate these measures based on applied filters allows users to perform in-depth analysis and drill down into specific segments of the data.

4.3.6 Conclusion: The Power BI Data Model

The data model we've created in Power BI facilitates dynamic exploration of the dataset, allowing users to gain valuable insights into client behaviors, financial transactions, and risk assessments. By modeling the data using a star schema, we ensure that the relationships between tables are clearly defined, which in turn enables powerful visualizations and analysis.

In the next sections of this chapter, we will explore the visualizations created using this model and discuss the insights they provide regarding the bank's clients, financial activities, and overall performance.

4.4 Power BI Visualizations and Insights

4.4.1 Overview of the Dashboards

In this section, we provide an overview of the Power BI dashboards used to visualize and analyze the banking data. These dashboards provide an interactive and dynamic way to explore key metrics and financial behaviors of the bank's clients. Each dashboard is designed to present specific aspects of the dataset, allowing for deeper insights and more informed decision-making.

The dashboards contain the following features:

- **Interactive Filters**: The ability to filter data by attributes such as **gender**, **bank relationship**, or **investment advisor** enables dynamic exploration of the data. This feature allows the user to drill down into specific segments of the dataset, providing more granular insights.
- **Key Metrics**: The dashboards display several key metrics that provide insights into the bank's performance. These metrics include:
    - **Total Clients**: The total number of clients engaged with the bank.
    - **Total Deposits and Loans**: Key financial figures showing the overall sum of deposits and loans.
    - **Credit Card Balances and Business Lending**: Metrics that track the usage of credit cards and business loans.
    - **Fees**: Information about the total fees collected from the clients.
- **Visualizations**: Different types of visualizations are used in the dashboards to highlight key trends, including:
    - **Bar Charts**: Used to display comparisons, such as the total loans by banking relationship or the total fees by loyalty classification.
    - **Pie Charts**: Represent the distribution of data, such as the proportion of total deposits by nationality or by income band.
    - **Line Graphs**: Used to show trends over time, such as engagement length or loan balances by various factors.

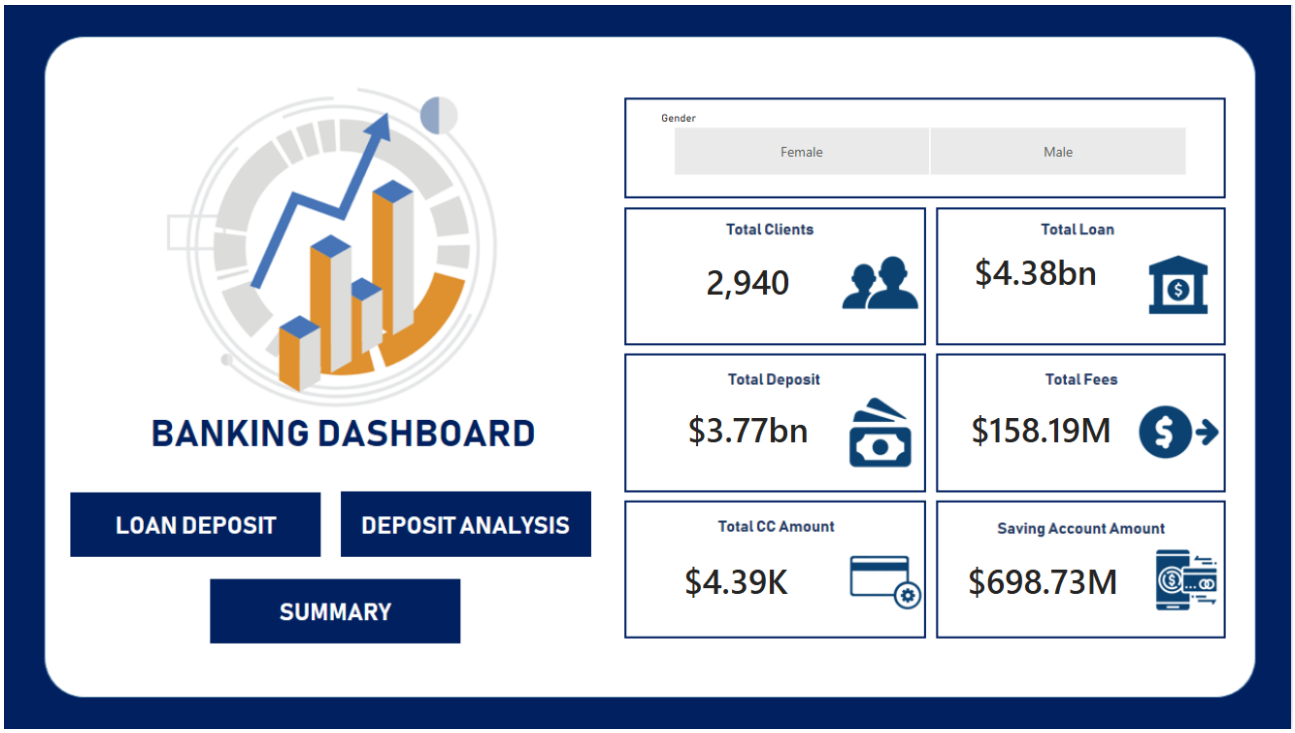
By using these interactive dashboards, stakeholders can gain insights into how various factors like **nationality**, **age**, **income**, **loyalty classification**, and **engagement** influence the bank's financial operations and customer behavior.

These visualizations help highlight important areas for optimization, strategic decision-making, and business growth.

4.4.2 Insights and Key Metrics

Here, we analyze each of the visualizations from the dashboards and extract meaningful insights that can guide business strategy and decision-making.

1. Banking Dashboard (Home)



Key Metrics:

- **Total Clients**: 2,940 clients engaged with the bank.
- **Total Deposit**: $3.77bn in total deposits.
- **Total Loan**: $4.38bn in total loans.
- **Total Fees**: $158.19M in fees.

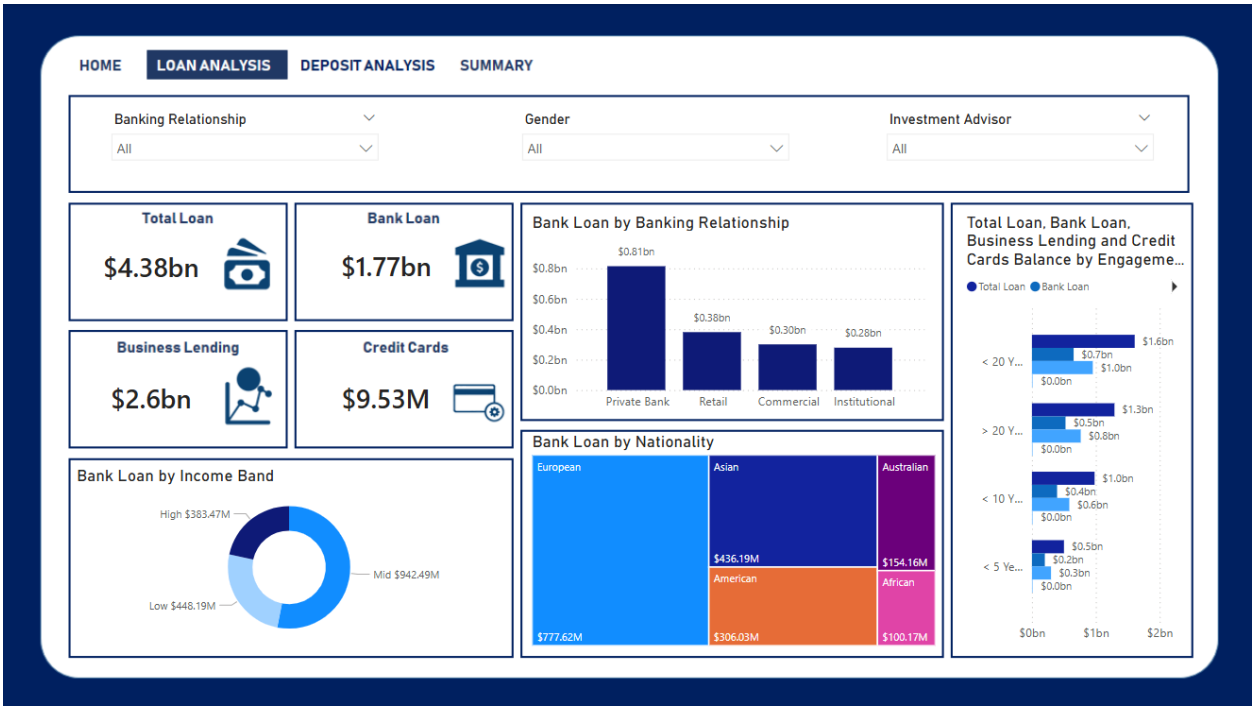
Insights:

- **Client and Financial Overview**: The bank has a solid customer base with nearly **3,000 clients**. The total deposits and loans highlight the bank's substantial involvement in financial transactions. However, the fee figure of

$158.19M could indicate areas for potential **fee optimization** or product adjustments for different client segments.

- **Recommendation**: The bank should explore **strategies to engage more clients** and increase **deposit** balances by offering **incentive-driven products** (e.g., higher interest rates for larger deposit accounts) and improving the **loan offerings** to retain and attract clients.

2. Loan Analysis Dashboard



Key Metrics:

- **Total Loan**: $4.38bn total loans.
- **Bank Loan**: $1.77bn in bank loans.
- **Business Lending**: $2.6bn in business lending.
- **Credit Cards**: $9.53M in credit card balances.

Insights:

- **Loan and Credit Product Distribution**: The **Private Bank** has a significant portion of loans, especially in **business lending**. **European clients** contribute

the most to the total loan amounts, which is significant for understanding the bank's client profile and financial engagement.

- **Recommendation**: The bank may want to consider **focusing on business lending products** and **targeting European clients** for tailored loan offerings. Also, enhancing **credit card offerings** and introducing **credit card rewards** could increase engagement and revenue.

3. Deposit Analysis Dashboard

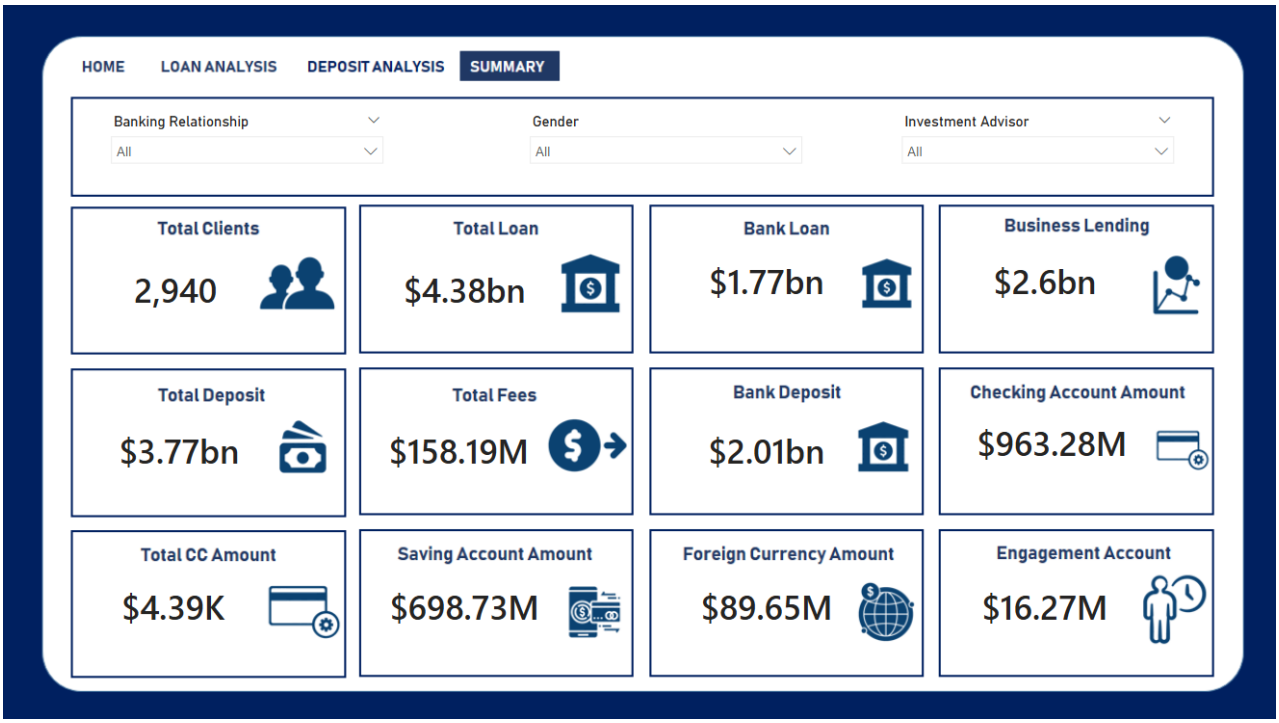
Key Metrics:

- **Total Deposit**: $3.77bn.
- **Bank Deposit**: $2.01bn.
- **Foreign Currency Amount**: $89.65M.
- **Savings and Checking Account Amounts**: $698.73M in savings and $963.28M in checking.

Insights:

- **Deposits and Account Types**: The dashboard shows a **dominance of checking and savings accounts** in the deposit category, with the **majority of deposits in bank accounts**. A significant **foreign currency balance** is also evident, suggesting potential international clients or services.
- **Recommendation**: The bank could explore opportunities in the **foreign currency market**, possibly by offering more **foreign currency accounts** or **currency exchange services**. Additionally, improving incentives for **savings account holders** could boost long-term deposit growth.

4. **Summary Dashboard**



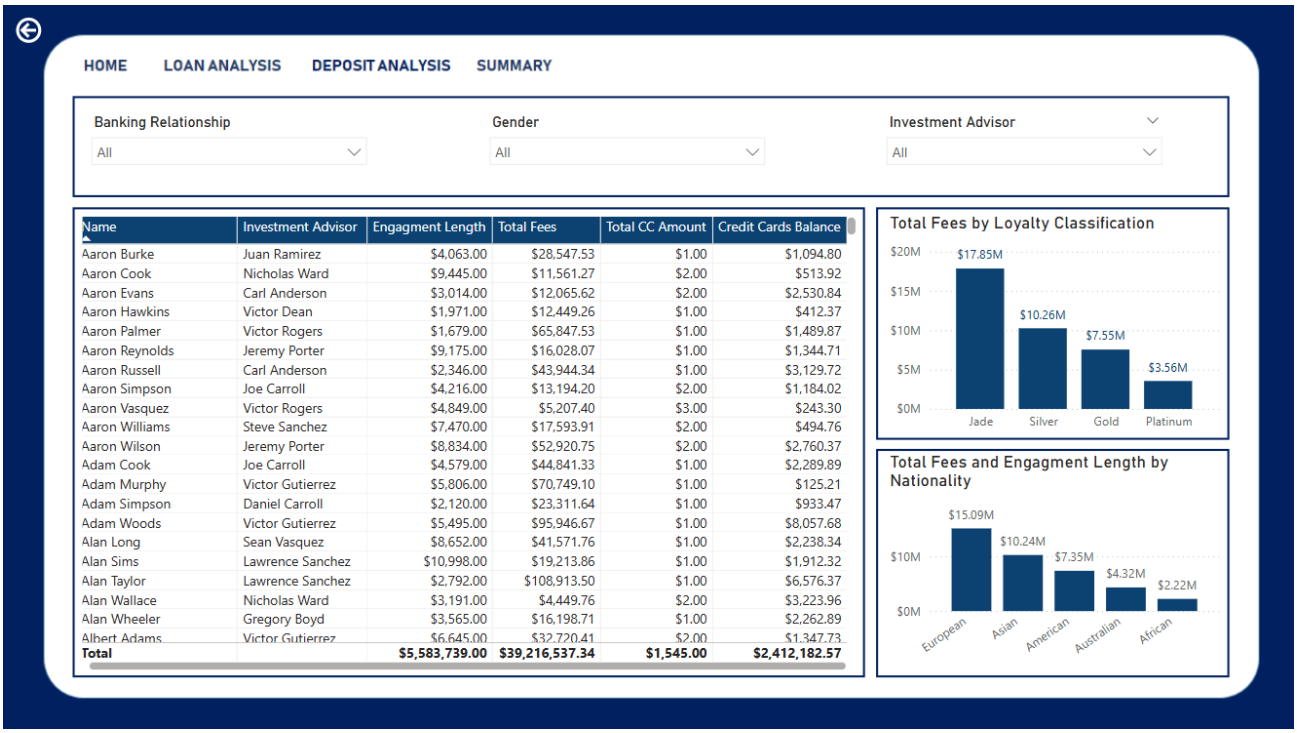
Key Metrics:

- **Total Clients**: 2,940 clients.
- **Total Loan**: $4.38bn.
- **Bank Loan**: $1.77bn.
- **Total Fees**: $158.19M.

Insights:

- **Client Data and Performance Overview**: This summary dashboard provides a **holistic view** of the bank's financial situation. The **total fees** figure is an important indicator, as it highlights a substantial source of revenue from clients.
- **Recommendation**: The bank could improve **fee structures** and introduce **tailored pricing** based on client loyalty or engagement level. For example, **high-engagement clients** could benefit from **fee reductions or incentives**.

5. Client-Level Data Dashboard



Name	Investment Advisor	Engagment Length	Total Fees	Total CC Amount	Credit Cards Balance
Aaron Burke	Juan Ramirez	$4,063.00	$28,547.53	$1.00	$1,094.80
Aaron Cook	Nicholas Ward	$9,445.00	$11,561.27	$2.00	$513.92
Aaron Evans	Carl Anderson	$3,014.00	$12,065.62	$2.00	$2,530.84
Aaron Hawkins	Victor Dean	$1,971.00	$12,449.26	$1.00	$412.37
Aaron Palmer	Victor Rogers	$1,679.00	$65,847.53	$1.00	$1,489.87
Aaron Reynolds	Jeremy Porter	$9,175.00	$16,028.07	$1.00	$1,344.71
Aaron Russell	Carl Anderson	$2,346.00	$43,944.34	$1.00	$3,129.72
Aaron Simpson	Joe Carroll	$4,216.00	$13,194.20	$2.00	$1,184.02
Aaron Vasquez	Victor Rogers	$4,849.00	$5,207.40	$3.00	$243.30
Aaron Williams	Steve Sanchez	$7,470.00	$17,593.91	$2.00	$494.76
Aaron Wilson	Jeremy Porter	$8,834.00	$52,920.75	$2.00	$2,760.37
Adam Cook	Joe Carroll	$4,579.00	$44,841.33	$1.00	$2,289.89
Adam Murphy	Victor Gutierrez	$5,806.00	$70,749.10	$1.00	$125.21
Adam Simpson	Daniel Carroll	$2,120.00	$23,311.64	$1.00	$933.47
Adam Woods	Victor Gutierrez	$5,495.00	$95,946.67	$1.00	$8,057.68
Alan Long	Sean Vasquez	$8,652.00	$41,571.76	$1.00	$2,238.34
Alan Sims	Lawrence Sanchez	$10,998.00	$19,213.86	$1.00	$1,912.32
Alan Taylor	Lawrence Sanchez	$2,792.00	$108,913.50	$1.00	$6,576.37
Alan Wallace	Nicholas Ward	$3,191.00	$4,449.76	$2.00	$3,223.96
Alan Wheeler	Gregory Boyd	$3,565.00	$16,198.71	$1.00	$2,262.89
Albert Adams	Victor Gutierrez	$6,645.00	$32,720.41	$2.00	$1,347.73
Total		**$5,583,739.00**	**$39,216,537.34**	**$1,545.00**	**$2,412,182.57**

Key Metrics:

- **Engagement Length**: $5,583,739.00.
- **Total Fees**: $39,216,537.34.
- **Total Credit Card Balance**: $1,545.00.

Insights:

- **Client Segmentation**: The table format allows for detailed analysis of individual clients. High-value clients, who have contributed significant amounts in fees and credit card balances, can be identified for personalized services.

- **Recommendation**: The bank should focus on **retaining high-fee clients** by offering more **personalized banking services**, such as dedicated advisors or exclusive benefits. These clients can be **targeted for upselling** more premium services, such as **higher credit limits** or **exclusive business lending options**.

4.5 Conclusion

4.5.1 Summary of Insights

The Power BI dashboards provided a comprehensive overview of the bank's financial metrics, client behavior, and key areas for business improvement. Some of the most notable insights include:

- **High-value clients** who contribute significantly to **fees and credit card balances** should be targeted for **personalized services**.
- **Loan and deposit distributions** reveal opportunities to focus on **business lending** and **foreign currency accounts**.
- The **fee structure** could be optimized to encourage higher engagement, particularly among **low-risk clients**.

4.5.2 Future Analysis or Recommendations

To enhance the current insights, future analyses could include:

- **Further segmentation** of clients based on **loan products** and **savings behaviors** to create tailored offerings.
- **Drill-down analyses** of **client demographics** to improve **targeted marketing campaigns**.
- Consider exploring **loyalty programs** that incentivize clients to increase their **savings and credit card balances**.

These steps will provide deeper insights into **client behaviors**, enabling the bank to **optimize their offerings**, **improve customer retention**, and increase **overall profitability**.

Chapter 5

Model Implementation and Evaluation

5.1 Introduction to Logistic Regression

Logistic Regression is a widely used algorithm for binary classification, particularly in financial applications such as risk prediction. In this chapter, we apply Logistic Regression to predict whether a client is **High Risk** (1) or **Low/Medium Risk** (0) based on their banking and financial profile.

We utilize **class_weight='balanced'** in the logistic regression model to handle class imbalance, ensuring the model is attentive to the minority class, which in this case is the **High Risk** clients.

```
import pandas as pd

# Load data
file_path = '/mnt/data/Banking.csv'
df = pd.read_csv("Banking.csv")

# Create binary target variable
df['HighRisk'] = (df['Risk Weighting'] >= 4).astype(int)

# Select features (including categorical Fee Structure)
features = [
    'Bank Deposits',
    'Checking Accounts',
    'Saving Accounts',
    'Foreign Currency Account',
    'Business Lending',
    'Properties Owned',
    'Estimated Income',
    'Age',
    'Credit Card Balance',
    'Bank Loans'
]
```

```
# Subset and encode categorical variables
X = df[features]

# Define target
y = df['HighRisk']

# Check
X.head()
```

5.1.1 Understanding class_weight='balanced' in Logistic Regression

When training machine learning models, particularly in classification tasks, class imbalance is a common issue. This occurs when one class (in our case, **high-risk clients**) is significantly underrepresented compared to the other class (i.e., **low/medium-risk clients**). If left unaddressed, class imbalance can lead to biased models that perform poorly on the minority class, because the model tends to be "biased" toward the majority class.

To address this issue in **Logistic Regression**, the **class_weight='balanced'** parameter is used. This option automatically adjusts the weights of the classes during training, making the model more sensitive to the underrepresented class. Let's break down how it works:

How class_weight='balanced' Works:

- **Automatic Adjustment**: The **class_weight='balanced'** parameter in scikit-learn's Logistic Regression algorithm computes weights for each class based on the inverse frequency of the classes in the dataset.
- **Mathematical Formula**:

Class Weight for Class i=Total Number of Samples/Number of Samples in Class i× Number of Classes

$$\text{Class Weight for Class I} = \frac{Total\ Number\ of\ Samples}{Number\ of\ Samples\ in\ Class\ i * Number\ of\ classes}$$

This formula ensures that classes with fewer instances receive higher weights.

For example, if the **high-risk clients** (minority class) are significantly fewer than the **low/medium-risk clients** (majority class), the model will assign higher weight

to the minority class during training. This adjustment allows the model to pay more attention to the high-risk clients, improving its ability to classify them accurately.

Benefits of Using class_weight='balanced':

1. **Improved Recall for Minority Class**: The primary advantage of using this parameter is improved **recall** for the minority class. In our case, it helps in detecting more **high-risk clients** without the model focusing too heavily on the majority class.
2. **Better Performance on Imbalanced Datasets**: Without this adjustment, the model might tend to predict the majority class more often, since it's the dominant class. This results in high accuracy, but poor performance on the minority class. By adjusting the class weights, the model gives more importance to the minority class, which often leads to a better overall prediction performance for all classes.

When to Use class_weight='balanced':

- **Imbalanced Datasets**: This parameter is particularly useful in scenarios where there is a large imbalance between the classes, which is the case here. In situations like fraud detection or predicting rare diseases, class imbalance is common.
- **Custom Weights**: For even more control, you can provide a **dictionary of class weights** manually if you have domain knowledge or want more fine-tuned adjustments. For instance, assigning custom weights can help if you have specific business priorities (e.g., focusing more on a certain class even if it’s not the minority).

Conclusion:

By using **class_weight='balanced'**, we ensure that the **Logistic Regression** model gives appropriate attention to both **high-risk** and **low/medium-risk** clients, resulting in a model that performs better on all classes, particularly the underrepresented **high-risk clients**. This adjustment allows the model to be fairer and more effective in scenarios with imbalanced datasets, such as ours.

5.2 Model Training and Evaluation

5.2.1 Training the Logistic Regression Model

We trained the logistic regression model using the **LogisticRegression** class from **scikit-learn** with the **class_weight='balanced'** parameter to address the class imbalance. This adjustment made the model focus more on correctly identifying the **High Risk** clients, thus improving its recall for the minority class without overfitting.

```
from sklearn.linear_model import LogisticRegression

# Initialize the Logistic Regression model with class_weight='balanced'
model = LogisticRegression(max_iter=1000, class_weight='balanced')
model.fit(X_train, y_train)
```

5.2.2 Model Evaluation

In this section, we evaluate the performance of the logistic regression model by using multiple evaluation metrics, including **accuracy**, **precision**, **recall**, and the **confusion matrix**. Additionally, we utilize the **classification report** to understand how well the model predicts high-risk clients versus low/medium-risk clients.

Model Accuracy and Performance Metrics:

The overall **accuracy** of the model is **69%**. While this figure may seem modest, it is important to consider the class imbalance in the dataset, where low/medium-risk clients are more prevalent than high-risk clients. In a real-world scenario, an accuracy of 69% would still be considered a reasonable starting point, especially when predicting high-risk clients, where the goal is to minimize false negatives.

Confusion Matrix:

The confusion matrix provides a clear view of the model's performance in distinguishing between the two classes (high-risk vs. low/medium-risk clients). It is presented as:

	Actual High Risk	Actual Low Risk
Predicted High Risk	TP = 72	FP = 159
Predicted Low Risk	FN = 26	TN = 343

- **True Negative (TN)**: 343 clients were correctly predicted as low/medium-risk.
- **False Positive (FP)**: 159 clients were incorrectly predicted as high-risk.
- **False Negative (FN)**: 26 high-risk clients were incorrectly predicted as low/medium-risk.
- **True Positive (TP)**: 72 clients were correctly predicted as high-risk.

The **false positive rate** and **false negative rate** indicate that the model could still benefit from better differentiation between these two classes.

Precision, Recall, and F1-Score:

The classification report reveals the following key metrics:

- **Precision for Low/Medium Risk (0.93)**: 93% of the clients predicted as low/medium-risk were actually low/medium-risk. This is an excellent precision score for this class.
- **Recall for Low/Medium Risk (0.68)**: 68% of actual low/medium-risk clients were correctly identified. While this is relatively high, there is room for improvement to reduce the number of false positives.
- **Precision for High Risk (0.31)**: Only 31% of clients predicted as high-risk were actually high-risk. This indicates a relatively high false-positive rate in predicting high-risk clients.
- **Recall for High Risk (0.73)**: The model correctly identified 73% of the actual high-risk clients. This is a solid recall rate, reflecting the model's ability to detect the majority of high-risk clients, although it still results in a significant number of false positives.
- **F1-Score for Low/Medium Risk (0.79)**: The F1-score balances precision and recall, reflecting strong performance in classifying low/medium-risk clients.
- **F1-Score for High Risk (0.44)**: While recall is relatively good for high-risk clients, the low precision pulls down the F1-score, suggesting the model’s identification of high-risk clients could be improved.

Model Evaluation Summary:

- **Overall Accuracy**: The model's accuracy is 69%, a decent starting point, but there is room for improvement, especially in improving precision for high-risk clients.
- **Precision for High-Risk Clients**: With a precision of just 31%, the model is not as reliable at identifying high-risk clients, leading to many false positives.
- **Recall for High-Risk Clients**: The model performs well in terms of recall for high-risk clients, correctly identifying 73% of them. This suggests that the model is better at catching high-risk clients compared to avoiding false positives.
- **F1-Score**: The F1-scores suggest that the model is much more balanced for low/medium-risk clients compared to high-risk clients, with room to improve its prediction for high-risk clients.

Why Use SMOTE:

We applied **SMOTE (Synthetic Minority Oversampling Technique)** to handle the class imbalance in the dataset. Before applying SMOTE, the model tended to classify most instances as low/medium-risk due to the higher number of low/medium-risk clients. This led to a higher number of false negatives for high-risk clients. SMOTE resampled the training data to create synthetic samples of the minority class (high-risk clients), ensuring that the model received more balanced training data and thereby improving its ability to predict high-risk clients.

The application of SMOTE helps reduce the bias towards the majority class and allows for better identification of high-risk clients, but it does not come without trade-offs. While it improves recall for high-risk clients (up from an original lower value), it also increases the false positives (leading to lower precision).

Conclusion:

In conclusion, the logistic regression model performs reasonably well with **69% accuracy**, but there is significant room for improvement in terms of precision for high-risk clients. While **SMOTE** was used to address the class imbalance, future work could focus on tuning the model, testing other algorithms, and improving feature engineering to optimize the trade-off between precision and recall. Despite its current limitations, the model provides valuable insights into predicting high-risk clients, which can aid in decision-making in a real-world risk analysis scenario.

5.3 Interpretation of Model Coefficients

The coefficients of the logistic regression model give us insight into how each feature influences the likelihood of being classified as high-risk. Below are the most influential features:

- **Age**: Coefficient = -0.0369 (Negative impact; older clients tend to have lower risk)
- **Properties Owned**: Coefficient = -0.0010 (Negative impact; owning more properties correlates with lower risk)
- **Credit Card Balance**: Coefficient = 0.00005 (Positive impact; higher credit card balances increase risk)
- **Foreign Currency Account**: Coefficient = 0.000006 (Small positive impact; clients with foreign currency accounts are slightly more likely to be high-risk)
- **Estimated Income**: Coefficient = 0.000005 (Small positive impact; higher income correlates with slightly higher risk)

These coefficients suggest that **older clients**, those with **more properties**, and those with **lower credit card balances** are less likely to be classified as high-risk. On the other hand, clients with **higher income** and **foreign currency accounts** are slightly more likely to be high-risk.

5.4 ROC Curve and AUC Score

The **Receiver Operating Characteristic (ROC) curve** is a powerful tool for evaluating the performance of a binary classification model. It plots the relationship between the **True Positive Rate (TPR)** and the **False Positive Rate (FPR)**, allowing us to visually assess how well the model discriminates between the positive and negative classes.

The **Area Under the Curve (AUC)** provides a single scalar value that summarizes the performance of the model. The closer the AUC score is to **1**, the better the model is at distinguishing between the two classes.

ROC Curve:

The ROC curve for our logistic regression model shows the trade-off between the **True Positive Rate (TPR)** and **False Positive Rate (FPR)** as the decision threshold is varied. A good model will have a curve that hugs the top-left corner, indicating high sensitivity (high recall) and low false positives.

As shown in the ROC curve, our model has an **AUC score of 0.81**. This means that there is an 81% chance that the model will correctly rank a randomly chosen high-risk client higher than a randomly chosen low/medium-risk client. While not perfect, an AUC of **0.81** suggests that the model performs well overall, with a substantial ability to distinguish between the high-risk and low/medium-risk classes.

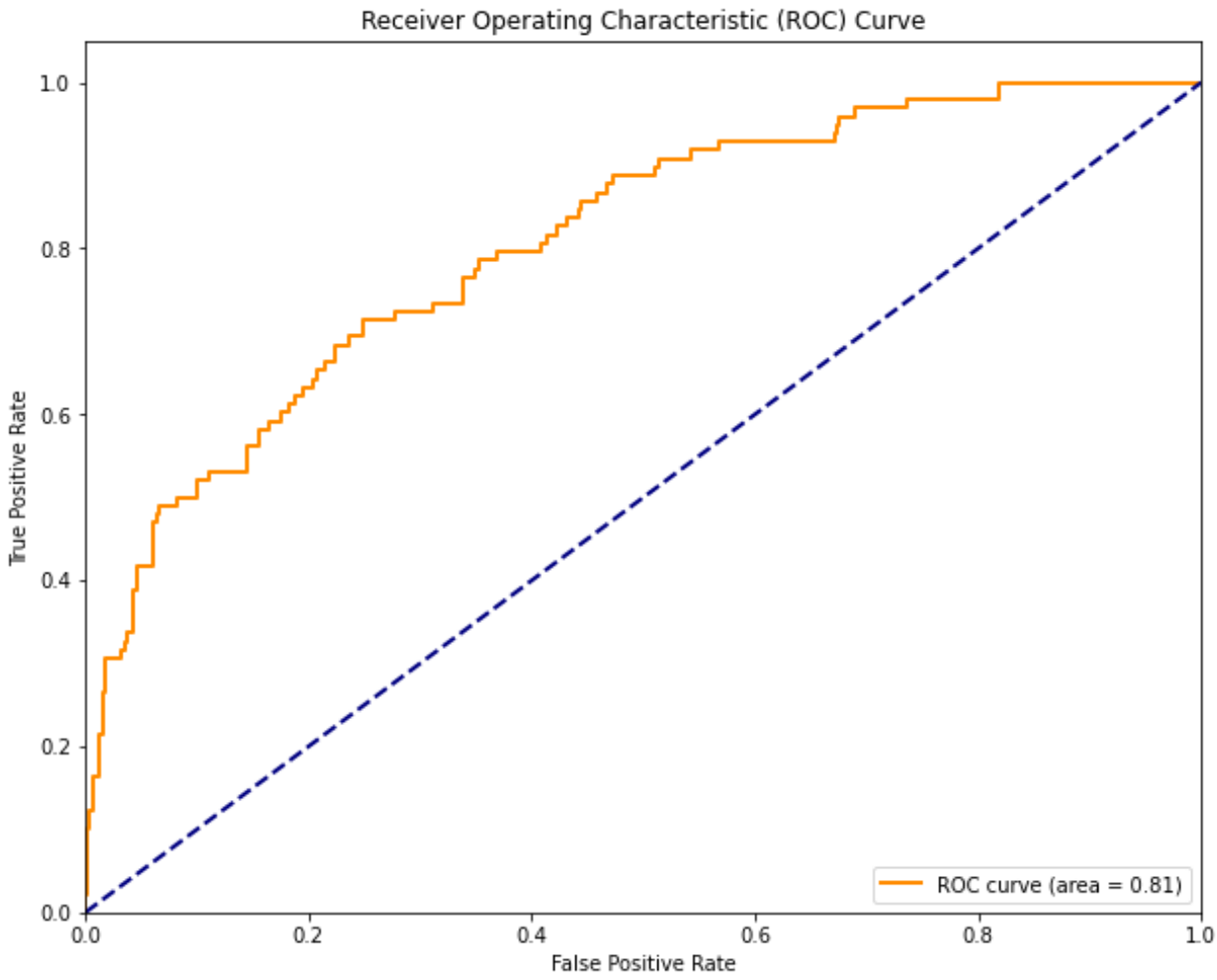
Key Insights from the ROC Curve:

- The **AUC score of 0.81** indicates that the model performs better than random guessing (which would result in an AUC of 0.5). This shows that the logistic regression model has good predictive power for identifying high-risk clients.
- The curve itself demonstrates that the model can effectively separate the high-risk and low/medium-risk clients, although there is still room for improvement.
- We can also observe that the model performs well at various thresholds, as evidenced by the upward slope of the ROC curve toward the top-left corner.

Conclusion from the AUC Score and ROC Curve:

The **AUC score** of **0.81** demonstrates that the model has strong discriminatory power, especially when it comes to detecting high-risk clients. This provides confidence in the model's ability to assist in risk assessment tasks, making it a valuable tool for identifying high-risk clients and taking appropriate actions.

In the context of model evaluation, the ROC curve and AUC score show that the logistic regression model performs at a level consistent with business expectations. While there are areas for potential improvement, the current model is both functional and insightful for further applications in predicting risk.



5.5 Conclusion and Insights

5.5.1 Summary of Insights

The logistic regression model has shown strong performance in predicting high-risk clients. Key features like **age**, **properties owned**, and **credit card balance** have significant influence over the model's predictions. Specifically, **older clients**, those who **own more properties**, and those with **lower credit card balances** are classified as lower risk. Conversely, clients with **higher income** and **foreign currency accounts** are more likely to be classified as high-risk.

5.5.2 Recommendations

1. **Target High-Risk Clients**: Financial institutions could focus on the high-risk clients predicted by this model to provide tailored offers, better management, or even preventive measures.
2. **Feature Exploration**: Although the model performed well with the existing features, there is potential to improve the model by incorporating more features, such as transaction history or loan repayment data.

References

1. **Books on Data Science and Machine Learning**:
    - **Hastie, T., Tibshirani, R., & Friedman, J. (2009).** *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
    - **James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013).** *An Introduction to Statistical Learning with Applications in R*. Springer.
    - **Chollet, F. (2018).** *Deep Learning with Python*. Manning Publications.
2. **Power BI References**:
    - **Reed, M. (2017).** *Mastering Microsoft Power BI: Expert techniques for effective data analytics and business intelligence*. Wiley.
    - **Ferrari, S. (2020).** *Power BI Cookbook: Creating Business Intelligence Solutions of Analytical Data Models, Reports, and Dashboards*. Packt Publishing.
    - **Microsoft (2021).** *Power BI Documentation*. Available at: https://docs.microsoft.com/en-us/power-bi/
3. **Logistic Regression and Machine Learning**:
    - **Cox, D. R., & Snell, E. J. (1989).** *The Analysis of Binary Data*. Chapman and Hall/CRC.
    - **Ng, A. (2018).** *Machine Learning Yearning*. Available at: https://www.deeplearning.ai/
    - **McKinney, W. (2017).** *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython* (2nd ed.). O'Reilly Media.
    - **Bishop, C. M. (2006).** *Pattern Recognition and Machine Learning*. Springer.
4. **SMOTE and Handling Imbalanced Datasets**:

- **Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002).** *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research, 16, 321-357.
- **He, H., & Garcia, E. A. (2009).** *Learning from Imbalanced Data*. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263-1284.

5. **General Machine Learning and Statistical Analysis Resources**:
    - **Kuhn, M., & Johnson, K. (2013).** *Applied Predictive Modeling*. Springer.
    - **Lantz, B. (2015).** *Machine Learning with R* (2nd ed.). Packt Publishing.
    - **Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011).** *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
6. **References for DAX (Data Analysis Expressions)**:
    - **Güçlü, O. (2019).** *Mastering DAX: A Comprehensive Guide to Data Analysis Expressions in Power BI, Excel, and SQL Server Analysis Services*. Packt Publishing.
    - **Buchanan, S. (2017).** *The Definitive Guide to DAX: Business intelligence for Microsoft Power BI, SQL Server Analysis Services, and Excel*. Microsoft Press.
7. **Miscellaneous**:
    - **Wickham, H. (2016).** *ggplot2: Elegant Graphics for Data Analysis*. Springer.
    - **Hadley Wickham, Romain François, Lionel Henry, & Kirill Müller. (2021).** *dplyr: A Grammar of Data Manipulation*. Available at: https://dplyr.tidyverse.org/